

CCL6546-K26

Flow Like a Pro Lab Overview

Lab Guide for CCL6546 - Flow Like a Pro

Welcome

Welcome to Flow Like a Pro! In this hands-on lab, we'll dig into how Flow Designer really handles data behind the scenes. You'll learn when pill values refresh (and when they don't), how GlideRecord data can quietly go stale, and how to debug issues that slow you down. By the end, you'll have a stronger grasp of how to build reliable, real-time flows that do exactly what you intended — no surprises.

Estimated Duration: ~65 minutes (including time for Q&A / discussion)

Agenda

- Set Up Your Lab Environment (~5 min)
- **Section 1:** Data Pill Behavior (~15 min)
- **Section 2:** GlideRecord Behavior: Subflow vs. Action (~20 min)
- **Section 3:** Runtime Values and Debugging (~20 min)
- **Wrap-Up & Discussion** (~5 min)

Set Up Your Lab Environment

Estimated Time: 5 minutes

- 1 Log in to your assigned lab instance.
- 2 In the application navigator, type Flow Designer and press Enter.
- 3 Confirm that Flow Designer opens and you can see the home screen.
- 4 Confirm that the prebuilt flows and subflow are listed:

Use the *Name* filter to search for flows with "CCL" in the flow name.

The screenshot shows the Workflow Studio interface. The 'Flows' tab is selected in the top navigation bar. A filter dropdown is open, showing 'contains' and 'CCL' entered. The table lists several flows with 'Global' as the application and 'Published' as the status. The right sidebar shows recent updates for 'CCL - Broken Flow', 'Sub-Production tasks', and 'Production tasks'.

Name	Application	Status	Active	Updated	Updated by
CCL - Broken Flow	Global	Published	true	2026-04-20 16:33:20	system
Sub-Production tasks	Global	Published	true	2026-04-26 17:32:52	admin
Production tasks	Global	Published	true	2026-04-20 16:33:24	system
	Global	Published	true	2026-04-20 16:33:22	system
	Global	Published	true	2026-04-20 16:33:17	system

Using the 'Name' filter in the Flows listing

- Flows
 - CCL - Blocking Demo
 - CCL - Breaking Flow
 - CCL - Non-Blocking Demo
 - CCL - Refresh Demo
 - CCL - Subflow vs Action Demo

Workflow Studio

Homepage Operations Integrations

Playbooks **Flows** Subflows Triggers Actions Decision tables

New

Flows 5
Last refreshed 2m ago.

Name	Application	Status	Active	Updated	Updated by
CCL - Blocking Demo	Global	Published	true	2026-04-20 16:33:20	system
CCL - Broken Flow	Global	Published	true	2026-04-26 17:32:52	admin
CCL - Non-Blocking Demo	Global	Published	true	2026-04-20 16:33:24	system
CCL - Refresh Demo	Global	Published	true	2026-04-20 16:33:22	system
CCL - Subflow vs Action Demo	Global	Published	true	2026-04-20 16:33:17	system

Showing 1-5 of 5

20 rows per page

Pick up where you left off

- CCL - Broken Flow
Last updated: 34 min. ago by Sys...
- Sub-Production tasks
Last updated: 5 months ago by Sys...
- Production tasks
Last updated: 6 months ago by Sys...

Latest updates

- System Administrator modified CCL - Broken Flow 34 min. ago
- System Administrator modified Sub-Production tasks

The flows in this lab

- Subflows
 - CCL - Subflow Log
 - CCL - Subflow Update

Workflow Studio

Homepage Operations Integrations

Playbooks Flows **Subflows** Triggers Actions Decision tables

New

Subflows 2
Last refreshed just now.

Name	Application	Status	Active	Updated	Updated by
CCL - Subflow Log	Global	Published	true	2026-04-20 16:33:27	system
CCL - Subflow Update	Global	Published	true	2026-04-20 16:33:15	system

Showing 1-2 of 2

20 rows per page

Pick up where you left off

- CCL - Broken Flow
Last updated: 34 min. ago by Sys...
- Sub-Production tasks
Last updated: 5 months ago by Sys...
- Production tasks
Last updated: 6 months ago by Sys...

The subflows in this lab

- Actions
 - CCL - Action Hold Thread
 - CCL - Action Log

Workflow Studio

Homepage Operations Integrations

Playbooks Flows Subflows Triggers **Actions** Decision tables

New

Actions 2
Last refreshed just now.

Name	Application	Active	Updated	Updated by
CCL - Action Hold Thread	Global	true	2026-04-20 16:33:08	system
CCL - Action Log	Global	true	2026-04-20 16:33:11	system

Showing 1-2 of 2

20 rows per page

Pick up where you left off

- CCL - Refresh Demo
Last updated: 19 h. ago by Sys...
- CCL - Broken Flow
Last updated: 20 h. ago by Sys...
- Sub-Production tasks
Last updated: 5 months ago by Sys...

The actions in this lab

 If you don't see the flows, confirm you are logged in with the correct lab credentials and have the `flow_designer` role.

Section 1: Data Pill Behavior

In this section, you'll build and run flows that demonstrate how data pills behave differently depending on whether instructions are blocking or non-blocking, and see how subflows change data behind the scenes.

Estimated Time: 15 minutes

Exercise 1.1: Blocking vs Nonblocking Instructions - Data Pill Behavior

Objective: Understand how data pill values behave when flow actions use blocking vs. non-blocking instructions, and observe that pills reflect the value at the time they were resolved.

Estimated Time: 20 minutes

Context

In Flow Designer, a **blocking** instruction pauses execution until it completes before moving to the next action.

A **non-blocking** instruction kicks off work but allows the flow to continue immediately. This distinction directly affects when your data pills get their values — and whether downstream actions see updated data.

Actions — Blocking Demo

1 Open the flow "CCL - Blocking Demo" in Flow Designer.

2 Review the flow structure. Notice it has the following actions:

Action	Type	Description
Trigger	Record Created	Fires when an Incident is created where <i>Short Description</i> = CCL - Blocking Demo
1	Log Message	Logs the short description value of the trigger Incident record
2	Subflow (Blocking): CCL - Subflow Update	Calls CCL - Subflow Update — updates the short description on the Incident to append a new line "Updated by subflow". Runs with <i>wait_for_completion</i> = true.
3	Action: CCL - Log Priority	A custom action that receives the trigger Incident record and logs its short description value.
4	Subflow: CCL - Subflow Log	Calls a second subflow that takes the trigger Incident record as input and logs "CCL - Subflow log" + the short description value.

3

Before running the flow, predict:

The subflow in Action 2 adds a new line to the short description. Action 2 runs as blocking, so it completes before Action 3. Will the **CCL - Action Log** action in Action 3 show the **original** short description or the **updated** short description with a new line?

4

Think about it:

Action 3 is a custom action (not a subflow). How does an action receive the record — as a fresh database fetch or an in-memory object?

5

Click Test to run the flow.

Use an existing Incident record as input, select *Run test in the background*, and click **Run Test**. In this example 'INC0000001' is used.

Test flow

Run your flow to make sure it has no errors before you activate it. When the test finishes running, check the execution details to see each action's configuration, runtime values, and the log messages for any errors that occurred.

* Incident Record

☒ Run test in background

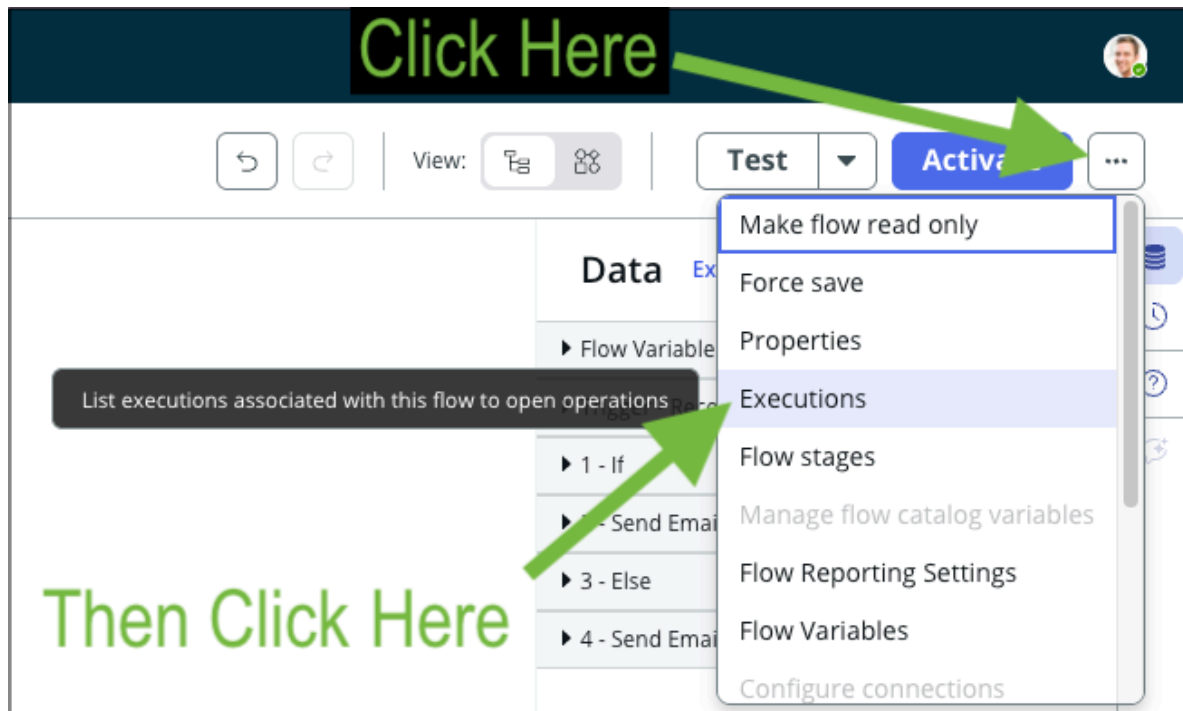
Test

6

Once the flow completes, open the **Execution Details**.

Close the Test dialog box. Click on the **More Actions menu** button to the right of the **Activate** button. The **More Actions menu** button is identified with ellipsis.

Select the **Executions** option to open the list of executions for this flow and then select the most recent entry.



More Actions menu > Executions

7

A note about testing flows with the Test button:

The Test button in Flow Designer runs a flow either in the foreground or background, controlled by the *Run test in background* checkbox. This distinction matters more than it might seem.

Running a flow **in the foreground** executes it within your current user session, which can introduce subtle differences in behavior — the flow may not run as the correct user or reflect the same conditions as a real trigger. Running **in the background** ensures the flow executes as the intended user and behaves exactly as it would during a normal trigger event.

The flows in this lab — such as **CCL - Subflow vs Action Demo** — have their trigger defined as *Run flow in background*. To match that trigger definition when using the Test button, make sure the *Run test in background* checkbox is checked.

One final note: the Test button **bypasses trigger evaluation entirely**. To test whether trigger conditions are working correctly, the flow must be activated and a record manually created or updated to satisfy the trigger conditions.

8

Compare the log outputs from Action 1 and Action 3.

1		Log info	Core Action	Completed	2026-04-26 18:59:20
Configuration Details					
VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE		
Level	info	info	Choice		
Message	CCL - Blocking Demo: Can't read email	CCL - Blocking Demo: Trigger - ... ▶ ... ▶ Short descri...	String		

Action 1

3

 CCL - Action Log

Completed

2026-04-26 18:59:20

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Record	INC0000001 	Trigger - Reco... ▶ Incident Re...	Reference

Output Data

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Action Status	{"Action Status":{"code":0,"message":"Success"}}		Object
Don't Treat as Error	true	true	True/False

No Logs

 Steps

 Step 1 - Log step [Log]

Completed

2026-04-26 18:59:20

Integration Metadata

Step Configuration

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Log Level	info	info	Choice
Log Message	CCL - Action Log: Can't read email + Subflow added a new line to short description	CCL - Action Log: action ▶ ... ▶ Short description	String

Action 3, with 'Steps' expanded

-  When *Wait for Completion* is checked, you are using a blocking instruction - the subflow runs synchronously on the same thread as the main flow.

After the subflow finishes and commits its changes to the database, the Flow Engine automatically re-fetches every record input and writes the refreshed GlideRecord back into the flow's runtime values. This refresh happens once the subflow completes, before the next action runs.

This is why action 3 (Action Log) and action 4 (Subflow Log) both display the updated short description value - the main flow's in-memory record was replaced with a fresh DB copy the moment the subflow returned.

Actions — Non-Blocking Demo

1 Now open the flow CCL - Non-Blocking Demo.

This flow has the same structure, except **Action 2 calls the subflow as non-blocking** *wait_for_completion = false*.

2 Run the test again, this time with INC0000002

Repeat the testing procedure from the previous example.

Test flow

Run your flow to make sure it has no errors before you activate it. When the test finishes running, check the execution details to see each action's configuration, runtime values, and the log messages for any errors that occurred.

* Incident Record ✕ ▼ 🔍 + ℹ

☒ Run test in background ℹ

Cancel Run Test

Test Dialog

3

Compare the execution details.

1  **Log info** Core Action **Completed** 2026-04-26 19:15:57

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Level	info	info	Choice
Message	CCL - Non-Blocking Demo: Network file shares access issue	CCL - Non-Blocking Demo: Trigger - ... ▶ ... ▶ Short descri...	String

Action 1

▼ 2  **CCL - Subflow Update** [Open Subflow Context](#) **Completed** 2026-04-26 19:15:57

1  **Update Record** Core Action **Completed** 2026-04-26 19:15:57

2  **Log info** Core Action **Completed** 2026-04-26 19:15:57

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Level	info	info	Choice
Message	CCL - Subflow Update: Network file shares access issue + Subflow added a new line to short description...	CCL - Subflow Update: Input ▶ ... ▶ Short description	String

Action 2, Subflow

3  **CCL - Action Log** **Completed** 2026-04-26 19:15:57

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Record	INC0000002 @	Trigger - Reco... ▶ Incident Re...	Reference

Output Data

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Action Status	{"Action Status":{"code":0,"message":"Success"}}		Object
Don't Treat as Error	true	true	True/False

No Logs

▼ **Steps**

 Step 1 - Log step [Log] **Completed** 2026-04-26 19:15:57

Integration Metadata

Step Configuration

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Log Level	info	info	Choice
Log Message	CCL - Action Log: Network file shares access issue	CCL - Action Log: action ▶ ... ▶ Short description	String

Action 3, Custom Action

▼ 4

CCL - Subflow Log

Open Subflow Context

Completed

2026-04-26 19:15:57

1

Log info

Core Action

Completed

2026-04-26 19:15:57

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Level	info	info	Choice
Message	CCL - Subflow Log: Network file shares access issue	CCL - Subflow Log: Input ... Short description	String

Action 4, Second subflow

When *Wait for Completion* is unchecked, the subflow is scheduled as a separate job on a different thread and transaction. The main flow does not wait for it, and the engine does not refresh the main flow's runtime values afterward.

As a result, any downstream action in the main flow keeps using the original in-memory GlideRecord and logs the original short description — even if the async subflow has already committed its update to the database. A downstream subflow still performs its own DB fetch, so it may or may not see the update depending on whether the async subflow has finished writing.

Takeaway

The *Wait for Completion* field determines whether an instruction is blocking or nonblocking, and this affects data availability to every downstream step.

Blocking: The subflow runs synchronously on the same thread. When it finishes, the engine re-queries every record input from the database and replaces the copy in the main flow's runtime values. Downstream actions and subflows will **see the updated record**.

Non-blocking: The subflow is queued on a separate thread/transaction; no refresh happens. Downstream actions see the **original in-memory record**. Downstream subflows will do their own DB fetch, so they see whatever is in the DB when they run.

Section 2: GlideRecord Behavior - Subflow vs. Action

In this hands-on session you'll observe a key difference in how GlideRecord data is passed to a subflow versus an action.

Estimated Time: 20 minutes

Exercise 2.1: Compare GlideRecord Values - Subflow vs Action

Objective: Understand that when you pass a GlideRecord to a subflow, it receives a reference (sys_id + table) and re-fetches from the database, while an action within the same flow uses the already-loaded in-memory record object — and how this affects data freshness.

Context

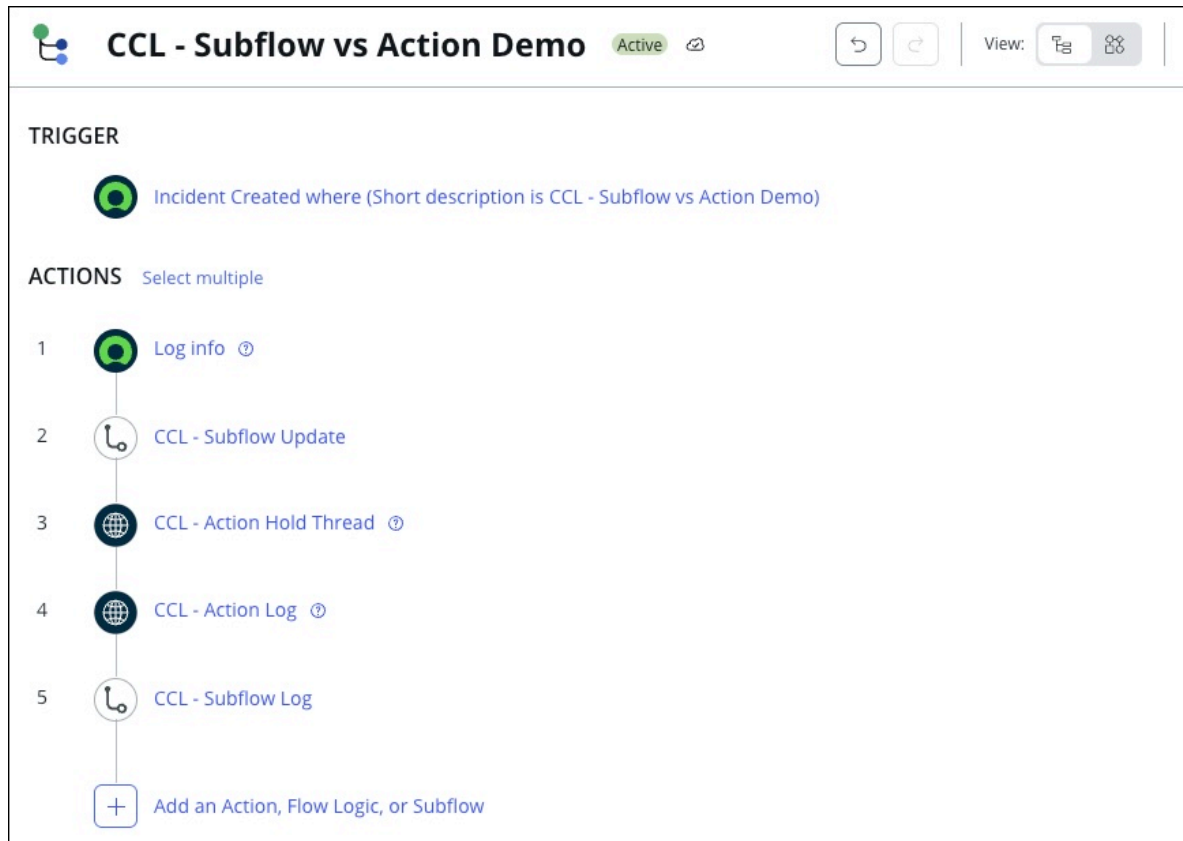
When you pass a record pill to a **subflow**, the Flow Engine makes a call to retrieve a fresh GlideRecord from the database. The subflow always starts with the **current DB state**.

When the same pill is used in an **action** within the same flow, the engine does a simple map lookup in the flow's runtime values and hands the action the **in-memory GlideRecord object**. The action does **no DB query**; it sees whatever the most recent refresh left in memory... meaning it could reflect stale data if the record was modified after the original query.

Walkthrough

1

Open the flow "CCL - Subflow vs Action Demo".



CCL - Subflow vs Action Demo

2

Review the structure:

Action	Type	Description
Trigger	Record Created	Fires when an Incident is created where <i>Short Description = CCL - Subflow vs Action Demo</i>
1	Log Message	Logs the initial short description of the trigger Incident record
2	Subflow: CCL - Subflow Update	Calls CCL - Subflow Update — updates short description to add a new line "Updated short description." Runs with <i>wait_for_completion = false</i>.
3	Action: CCL - Action Hold Thread	A custom action that pauses execution for 10 seconds , giving the non-blocking subflow enough time to complete
4	Action: CCL - Action Log	A custom action that takes the trigger Incident record and logs its short description
5	Subflow: CCL - Subflow Log	Calls a subflow that takes the trigger Incident record and logs its short description

3

Click Test to run the flow

This example will use INC0000003.

Test flow ×

Run your flow to make sure it has no errors before you activate it. When the test finishes running, check the execution details to see each action's configuration, runtime values, and the log messages for any errors that occurred.

* Incident Record × ▼ 🔍 + ℹ️

☒ Run test in background ℹ️

Cancel Run Test

Run the test in the background

4

Why the 10-second wait?

Action 2 fires the subflow as non-blocking, so the flow continues immediately.

Action 3 pauses for 10 seconds to ensure the non-blocking subflow has time to finish updating the database before we check the results.

```
Script
1 ☐ (function execute(inputs, outputs) {
2
3     gs.sleep(1000 * inputs.duration);
4     gs.log("CCL - Action Hold Thread - DONE");
5
6 })(inputs, outputs);
7
```

The script in the custom action

5

Observe the results:

- Wait 10 seconds after clicking 'Run Test' as the flow is still executing in the background even after the dialog closes. Then open the executions for this flow and open the most recent one.

- Action 1 (Log):** Logs the **original** *short description* — before any changes.

Configuration Details	
VARIABLE NAME	RUNTIME VALUE
Level	info
Message	CCL - Subflow vs Action Demo: Wireless access is down in my area

Action 1 Detail

- Action 3 (Action: CCL - Action Hold Thread):** This action uses `gs.sleep` to 'pause' the execution. This is an anti-pattern used for demonstration purposes only. `gs.sleep` does not put the flow to sleep, it is a busy wait, continuing to hold the active thread and preventing any other work to happen in it.
- Action 4 (Action: CCL - Action Log):** Even after waiting 10 seconds for the subflow to complete, this action still logs the **original** short description.
Why? Because it's an action that uses the **in-memory GlideRecord** from the trigger — it never re-fetches from the database.

4 **CCL - Action Log** Completed 2026-04-26 19:30:11

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Record	INC0000003	Trigger - Reco... ▶ Incident Re...	Reference

Output Data

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Action Status	{"Action Status":{"code":0,"message":"Success"}}		Object
Don't Treat as Error	true	true	True/False

No Logs

▼ Steps

Step 1 - Log step [Log] Completed 2026-04-26 19:30:11

Integration Metadata

Step Configuration

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Log Level	info	info	Choice
Log Message	CCL - Action Log: Wireless access is down in my area	CCL - Action Log: action ▶ ... ▶ Short description	String

Action 4 Detail

- **Action 5 (CCL - Subflow Log):** Because subflows receive a reference (table + sys_id) and **re-fetch the record from the database**, this subflow sees the **updated** short description.

EXECUTION DETAILS **CCL - Subflow** Open flow Open context record

▼ 2 CCL - Subflow Update

1 Upd

2 Log

Viewing log_message [string]

Rendered HTML Raw Text Code

CCL - Subflow Update: Wireless access is down in my area + Subflow added a new line to short description

Close

2026-04-27 21:50:13 675ms

2026-04-27 21:50:15 3852ms

2026-04-27 21:50:19 2ms

TYPE

Choice

String

CCL - Subflow Update: Wireless access is down in my area + Subflow added a new line to short description

CCL - Subflow Update: Input ▶ ... ▶ Short description

Action 5 Detail

6

Why does this matter?

This is one of the most common sources of "my flow isn't working" confusion. The 10-second wait proves the subflow had plenty of time to complete — yet the action **still** shows stale data.

The issue isn't timing; it's **how the record is passed (action vs subflow)**.

Key Differences Summary

	Subflow	Action
Record passed as	Fresh GlideRecord	In-memory GlideRecord object
Data freshness	Re-fetched from DB — always current	Uses cached in-memory value — may be stale
When to watch out	Generally safe for fresh data	Be cautious if record was modified elsewhere

Exercise 2.2: Use Look Up Record to Refresh Stale Data

1

Open the CCL - Refresh Demo flow.

Notice the additional **Look Up Record** action after the subflow, querying the same Incident by its Number.



An explicit lookup record

2

Run the test with INC0000004.

Test flow

×

Run your flow to make sure it has no errors before you activate it. When the test finishes running, check the execution details to see each action's configuration, runtime values, and the log messages for any errors that occurred.

* Incident Record

INC0000004

×

🔍

+

ⓘ

☒ Run test in background ⓘ

Cancel

Run Test

Run the test in the background

Now, the last log reflects the new updated short description value.

5

CCL - Action Log

Completed

2026-04-26 19:49:55

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Record	INC0000004 ⓘ	4 - Look Up ... ▶ Incident Rec...	Reference

Output Data

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Action Status	{"Action Status":{"code":0,"message":"Success"}}		Object
Don't Treat as Error	true	true	True/False

No Logs

▼ Steps

Step 1 - Log step [Log]

Completed

2026-04-26 19:49:55

Integration Metadata

Step Configuration

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Log Level	info	info	Choice
Log Message	CCL - Action Log: Forgot email password + Subflow added a new line to short description	CCL - Action Log: action ▶ ... ▶ Short description	String

Example execution

ⓘ The third log now shows the updated short description — because the Look Up Record performed a fresh database fetch.

Takeaway

When you need guaranteed fresh data in a downstream action, prefer passing the *sys_id* and using **Look Up Record** within that action to re-fetch, rather than relying on how the record pill is implicitly passed.

Section 3: Runtime Values and Debugging

Objective: Learn to launch Debug mode in Flow Designer, set breakpoints, and use the debugger to inspect runtime values step by step.

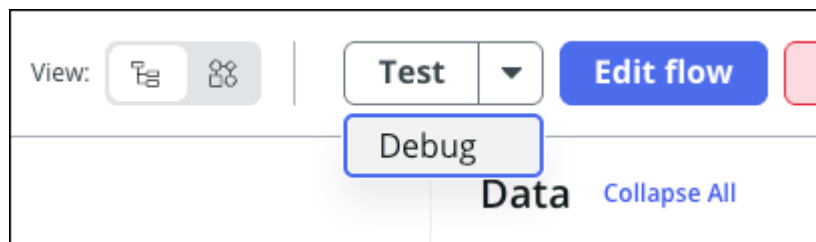
This section covers where runtime data lives and how to access it for debugging.

Estimated Time: 20 minutes

3.1: Launching Debug Mode

1 Open the "CCL - Blocking Demo" flow from the earlier exercises.

2 Click the dropdown arrow next to the Test button and select Debug.



The 'Debug' button is under the ▼

3 Enter your trigger inputs.

The input interface is the same as Test mode. For this example use INC0008112.

Debug flow ×

Set breakpoints to pause a flow at specific locations and then run it step-by-step to identify errors and unexpected results.

* Incident Record

INC0008112 × ▼

🔍 + ⌚

Cancel

Debug

Test with INC0008112

4

Click Debug. The flow opens in a new Workflow Studio tab in Debug Mode.

Input records use their current state, not their initial state — same behavior as Test mode.

-  No special role is required beyond Flow Designer access. If you can open Flow Designer, you can use the debugger. The Script Debugger role is only needed when stepping into the platform script debugger.

3.2: Navigating with Debug Controls

Use the five navigation controls in the top right of the debug tab to step through execution:



The five debugging controls

1. **Resume:** Runs until the next breakpoint (or to completion if none remain)
2. **Step Over:** Advances one step regardless of breakpoints; treats subflows as a single step
3. **Step Into:** Opens a subflow or script in its own debug tab; parent flow pauses
4. **Step Out:** Exits the current subflow and returns to the parent tab
5. **Skip All Breakpoints:** Runs the flow to completion, ignoring remaining breakpoints

i For **custom action scripts:** When paused at a script step, click **Step Into** (or the **Script Debug** inline link on the step) to open the platform script debugger. Step through the script line by line and inspect variables. Script outputs are surfaced back in the Flow Debugger after completion. **Requires the Script Debugger role.**

For **loops:** Set a breakpoint **inside** the loop body. Each **Resume** advances one iteration — inspect data after each pass. Once the early iterations look clean, remove the loop breakpoint and **Resume** to run through the rest.

3.3: Setting Breakpoints & Inspecting Runtime Values

1

In the debug tab, click the circle icon to the left of any action to set a breakpoint.

The circle fills in to indicate it is active.

TRIGGER

 Incident Created [Open current record](#)

ACTIONS

		Core Action	Debugging	2026-04-26 20:24:38												
1		Log info														
Configuration Details <table border="1"> <thead> <tr> <th>VARIABLE NAME</th> <th>RUNTIME VALUE</th> <th>CONFIGURATION</th> <th>TYPE</th> </tr> </thead> <tbody> <tr> <td>Level</td> <td>info</td> <td>info</td> <td>Choice</td> </tr> <tr> <td>Message</td> <td>CCL - Blocking Demo: Assessment : ATF Assessor</td> <td>CCL - Blocking Demo: Trigger - ... ▶ ... ▶ Short descri...</td> <td>String</td> </tr> </tbody> </table>					VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE	Level	info	info	Choice	Message	CCL - Blocking Demo: Assessment : ATF Assessor	CCL - Blocking Demo: Trigger - ... ▶ ... ▶ Short descri...	String
VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE													
Level	info	info	Choice													
Message	CCL - Blocking Demo: Assessment : ATF Assessor	CCL - Blocking Demo: Trigger - ... ▶ ... ▶ Short descri...	String													
Output Data <table border="1"> <thead> <tr> <th>VARIABLE NAME</th> <th>RUNTIME VALUE</th> <th>CONFIGURATION</th> <th>TYPE</th> </tr> </thead> <tbody> <tr> <td>Action Status</td> <td></td> <td></td> <td>Object</td> </tr> <tr> <td>Don't Treat as Error</td> <td>false</td> <td></td> <td>True/False</td> </tr> </tbody> </table>					VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE	Action Status			Object	Don't Treat as Error	false		True/False
VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE													
Action Status			Object													
Don't Treat as Error	false		True/False													
No Logs																
2		CCL - Subflow Update		Not Run												
3		CCL - Action Log		Not Run												

Initial debugging view with a breakpoint on action 2

 Good candidates for strategic breakpoints are subflow calls, conditional branches, and actions that write to records.

2

Click Debug / Resume to run the flow.

Execution pauses at the first breakpoint and an orange "Debugging" badge appears on the active action.

ACTIONS

Icon	ID	Name	Status	Timestamp
🔍	1	Log info	Completed	2026-04-26 20:24:38
🔴	2	CCL - Subflow Update	Debugging	2026-04-26 20:25:55

Subflow Input

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Wait For Completion	true	true	True/False
Record	552c48888c033300964f4932b03eb092 ⓘ	Trigger - Reco... ▶ Incident Re...	Reference

Subflow Output

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Context			Reference
Short Description			String

No Logs

Debugging paused on action 2

3

Review the Runtime & Configuration Values panel for the paused action.

- *Variable Name*: The variable from this action
- *Runtime Value*: What the action actually produced at runtime
- *Configuration*: What you configured (pill references, static values)
- *Type*: Data type (Reference, String, True/False, etc.)
- Compare Runtime Values to Configuration to identify mismatches.

ⓘ You can add or remove breakpoints at any time during an active session — no need to restart.

3.4: Hands-On — Debug the CCL - Broken Flow

Scenario: The **CCL - Broken Flow** should send a tailored email to the assigned engineer whenever an incident is created or updated. If the incident is urgent, the engineer's manager should be CC'd and the email should request prioritization. If not urgent, a standard notification goes to the engineer only.

Three problems have been reported: emails sometimes aren't sent at all, the wrong email type goes out on genuinely urgent incidents, and when emails do attempt to send, they fail to deliver.

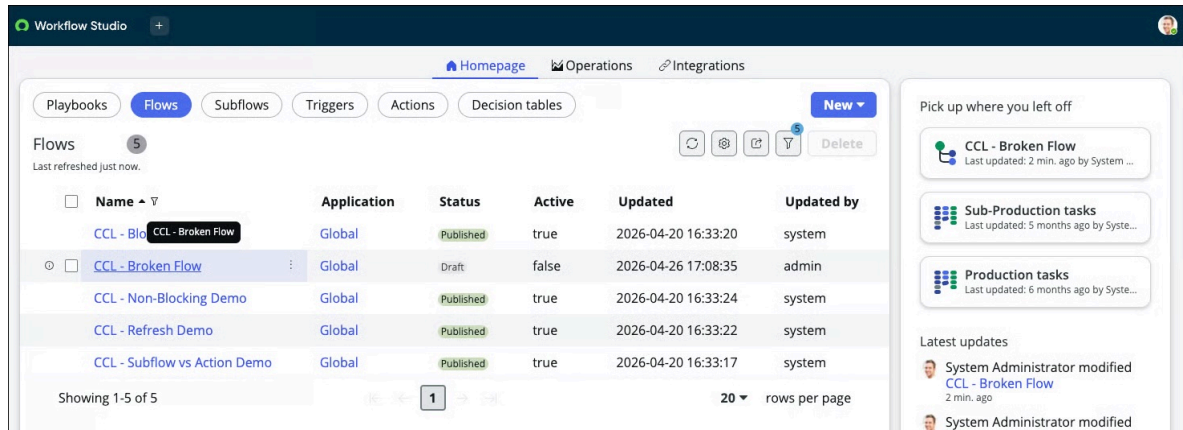
You have two real incidents to work with:

	INC0000054	INC0000049
Urgency	1 — High	1 — High
Priority	1 — Critical	2 — High
Assigned to	<i>(empty)</i>	Don Goodliffe
Surfaces	Bug 1	Bug 2 + Bug 3

3.4.1 — Find Bug 1 using INC0000054

1

Open CCL - Broken Flow in Workflow Studio

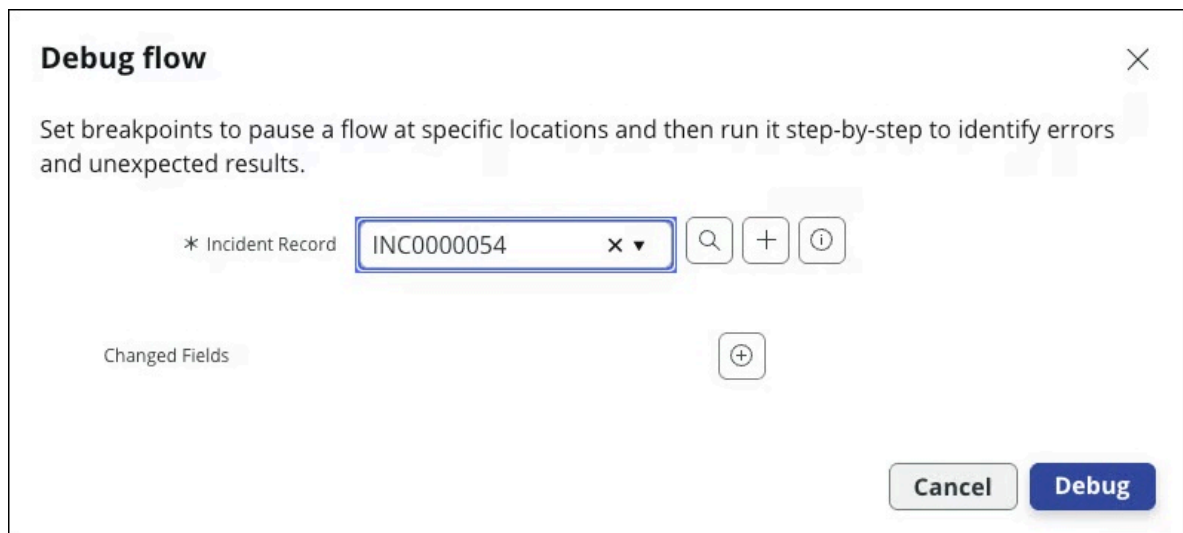


Select the CCL - Broken Flow

2

Click Test ▼ > Debug

1. Select **INC0000054** as your trigger record
2. Click **Debug**.



Debug with INC0000054

3

Set breakpoints on:

- Action 1 — the **If Urgent** condition
- Action 2 — the **then** Send Email (Urgent Incident Assigned)
- Action 4 — the **else** Send Email (New Incident Added)

Workflow Studio CCL - Broken Flow Flow • Global CCL - Broken Flow Flow execution • None

EXECUTION DETAILS CCL - Broken Flow

Debug Mode - Debugging Cancel flow Open flow Open context record

Set breakpoints and select any debugger button to begin

breakpoints hide Action details

FLOW STATISTICS Run as: System Administrator Open flow logs Debugging 2026-04-26 18:26:46 1ms

TRIGGER Incident Created or Updated Open current record

ACTIONS

1 If Urgent Flow Logic Debugging 2026-04-26 18:26:46 1ms

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Condition	1 - Critical=1	Trigger - Record ... > ... > Prio... =1	String
Condition Label	Urgent	Urgent	String

No Logs

2 Send Email Not Run

3 Else Flow Logic Not Run

4 Send Email Not Run

ERROR HANDLER

Add breakpoints on all actions in the flow

4

Notice!

The execution is already paused at Action 1.

5

Expand the Runtime & Configuration Values panel

- What field does the **Configuration** column show for Action 1?
- What is the **Runtime Value**?

TRIGGER

 Incident Created or Updated [Open current record ①](#)

ACTIONS

 1  If Urgent Flow Logic Debugging 2026-04-26 18:40:32 1ms

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Condition	1 - Critical=1	Trigger - Record ... > ... > Prio... =1	String
Condition Label	Urgent	Urgent	String

No Logs

This is a display issue only, the condition is successfully evaluated

 **INC0000054** has *Priority 1 — Critical*, so the condition evaluates true and the flow takes the then branch. Note the field being checked — we'll come back to this in Part 2.

The display here is showing a mix between Display Value and Value for the configuration panel.

6

Click Resume

Execution pauses at Action 2 — the urgent Send Email.

7

Expand the Runtime & Configuration Values panel on Action 2

- Find the **To** field. What is its **Runtime Value**?
- **Expected finding:** *null*. INC0000054 has no Assigned To populated.

8

This is Bug 1

The flow has no guard checking that Assigned To is populated before attempting to send. With a null recipient, the Send Email action will error or send to nobody.

Discussion: Where should the guard be added, and what should it check?

Answer: An If condition before the branching logic — Trigger > Record > Assigned to is not empty. The entire email logic sits inside the then branch. The else branch takes no action.

9

Click Skip All Breakpoints to end the session cleanly

ACTIONS				
▼ 1		If Urgent	Flow Logic	Evaluated - True
2		Send Email	Error	
Email validation failed: Email has no recipients.				

The final state of the flow execution

3.4.2 — Find Bugs 2 and 3 using INC0000049

1

Launch a new debug session

Select **INC0000049** as your trigger record.

- INC0000049 has *Urgency 1 — High* but *Priority 2 — High*. It is genuinely urgent. It has an assignee. The flow should take the urgent path and CC Don Goodliffe's manager.

2

Set breakpoints on actions 1, 2, and 4

3

Execution is already paused at action 1

Expand the Runtime Values panel on the If condition

- **Configuration** column: what field is the condition checking?
- **Expected finding:** *Trigger - Record > Priority = 1*
- **Runtime Value** for that field?
- **Expected finding:** *Priority = 2 — High = 1.*

If Urgent			
Flow Logic Debugging 2026-04-26 18:43:46 0ms			
Configuration Details			
VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Condition	2 - High=1	Trigger - Record ... ▶ ... ▶ Prio... =1	String
Condition Label	Urgent	Urgent	String
No Logs			

Paused on Action 1

- Click **Resume**. Note the flow takes the **Else** branch despite Urgency 1 — High.

4

This is Bug 2

- The condition checks *Priority* instead of *Urgency*. Click **Resume** — execution pauses at Action 4, the standard **Send Email**.

5

Expand the Runtime Values panel on Action 4

Locate the **To** field.

- **Runtime Value:** what does it show?
- **Type:** what data type is shown?
- **Expected finding:** Runtime Value shows a *sys_user* record reference — something like a *sys_id* or a user object. Type shows **Reference**, not **String**.

6

This is Bug 3

The To field data pill is configured to resolve to the Assigned To user *record* — a Reference to the *sys_user* table — rather than drilling into that user's **Email** field. The Send Email action needs a string email address, not a Reference object. The same misconfiguration exists on the CC field (Manager) in the urgent branch.

The distinction: The data pill *Trigger > Record > Assigned to* resolves to the user record. The correct pill is *Trigger > Record > Assigned to > Email* — one level deeper, resolving to the email address string on that user record.

4	Send Email	Debugging	2026-04-26 18:47:17	4ms
Configuration Details				
VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE	
To	9ee1b13dc6112271007f9d0efdb69cd0	Trigger - Reco... > ... > Assigne...	String	
Include Watermark	true	1	True/False	
BCC			String	

Assigned To is evaluated as a *sys_id*



Click Skip All Breakpoints to end the session

Debrief — all three bugs:

Bug	Test incident	Where found	What Runtime Values showed
Bug 1 — null recipient guard	INC0000054	Action 2 To field	Runtime Value = null
Bug 2 — wrong condition field	INC0000049	Action 1 condition	Configuration = Priority; wrong branch taken
Bug 3 — wrong data pill depth	INC0000049	Action 4 To field	Runtime Value = Reference object, Type = Reference not String

Bug 1 and Bug 2 would have been hard to diagnose from logs alone. Bug 3 might never appear in logs at all — the flow completes, an email is attempted, and it silently fails to deliver because the recipient field contained a record reference instead of an address. The Type column in the Runtime Values panel surfaces this in seconds.

3.5 Challenge — Fix and Verify

1 Open the Flow CCL - Broken Flow

- Select the tab in workflow studio.
- click **Edit Flow**.

2 Fix Bug 1

Update the trigger condition to include: *Trigger > Record > Assigned to is not empty*.

TRIGGER

Incident Created or Updated where (Short description contains Broken Demo, and Assigned to is not empty)

Trigger: Created or Updated

* Table: Incident [incident]

Short description contains Broken Demo or Assigned to is not empty

Bug 1 fix

3 Fix Bug 2

Open Action 1 (If Urgent). Change the condition field from *Priority* to *Urgency*, operator *is*, value *1 — High*. Save.

ACTIONS Select multiple

1 If

Condition Label: Urgent

* Condition 1: Trigger - Record... is 1 - High

Add another condition set(OR)

Delete Cancel Done

Bug 2 fix

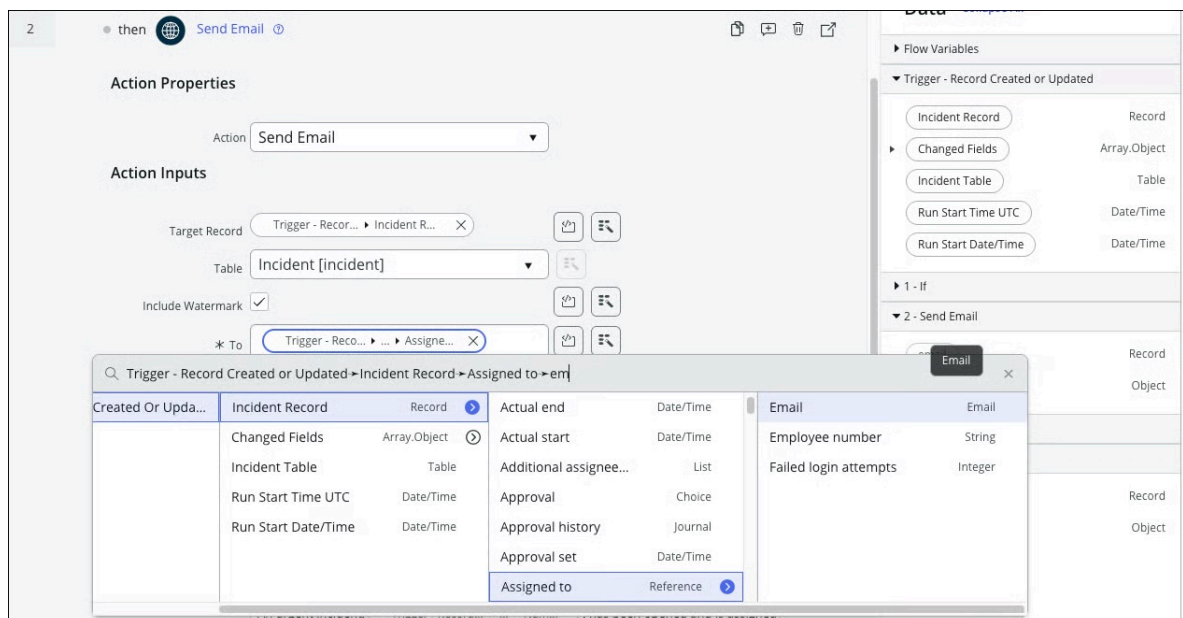
4

Fix Bug 3

Open Action 2 (urgent Send Email). Update the **To** data pill using the right hand data menu:

- Remove the current pill *Trigger > Record > Assigned to*
- Re-add it drilling one level deeper: *Trigger > Record > Assigned to > Email*
- Update the *CC* field the same way: *Trigger > Record > Assigned to > Manager > Email*

Open Action 4 (standard Send Email) and apply the same fix to the **To** field, this time do it by clicking the pill and continuing to drill down: *Trigger > Record > Assigned to > Email*



5

Verify all three fixes using INC0000049

Launch debug mode, select INC0000049.

- Set breakpoints on If Urgent, and the Send Email actions.
- **Resume** → If Urgent: Runtime Value = *1 — High*. Condition *true*, urgent branch taken.
- **Resume** → Action 2 and 4, Send Email: *To* Runtime Value = *an email address string*. Type = String.

Click **Skip All Breakpoints** to end the session

6

Verify using INC0000054

Launch a new debug session, select INC0000054.

- **Resume** → Assigned To guard: Runtime Value = null. Condition false, flow exits cleanly without attempting to send.

Click **Skip All Breakpoints** to end the session

Takeaway

The Flow Debugger replaces the old loop of **run** → **dig through logs** → **guess where it went wrong**.

Set breakpoints where you suspect problems, use the **Runtime Values** panel as your source of truth, and **Step Into** subflows and scripts to isolate nested logic.

Wrapup & Discussion

Estimated Time: 5 minutes + open discussion

Key Concepts Recap

- **Data pills are snapshots** — they capture the value at the time their originating action runs and do not auto-refresh.
- **Blocking vs. non-blocking** changes when subflows execute relative to each other, but in-memory record references remain stale regardless of timing.
- **Subflows re-fetch records** from the database (via *table* + *sys_id* reference), while **actions** in the same flow use the cached in-memory GlideRecord — leading to different values for the same record.
- **Look Up Record is your refresh button** — use it whenever you need guaranteed current data after a modification.
- **The CCL - Subflow vs Action Demo isolates the non-blocking use case** — even with a 10-second wait ensuring the subflow completes, the action still sees stale data. The difference is in **how** the record is passed, not **when**.

Discussion Questions

- Have you encountered stale data issues in your production flows? What was the symptom?
- How might you redesign an existing flow to avoid stale pill problems?
- When would you choose to use a subflow vs. an action, knowing how record data is passed differently?
- In the Subflow vs Action Demo, why does adding a wait *not* fix the stale data problem for the action?

Thank you for attending! We hope this lab gives you a clearer mental model of how data flows through Flow Designer. Happy building!

Reference Material

Workflow Center of Excellence:

<https://sn.works/coe> ↗

Flow Designer Data pill documentation:

<https://www.servicenow.com/docs/r/build-workflows/workflow-studio/data-population.html> ↗